Rankoya integrations: OAuth and credential flows guide

Nov 6, 2026 · @Favour Odili

1. Scope: two kinds of integration

Only Wix and Duda work like the five integrations you already have: Rankoya holds the user's credentials and pushes articles out. Lovable, Replit, Make, Zapier and RSS work the other way round: the outside tool pulls from Rankoya, so Rankoya must issue the credential (an API key, an OAuth token or a secret feed URL) instead of storing one.

Integration	Direction	Auth model	What a connection stores	Build effort
Wix	Rankoya pushes to Wix Blog	OAuth, through a Wix app the user installs	Wix app instance id (tokens are minted on demand)	Medium
Duda	Rankoya pushes to Duda Blog	Duda API username and password + site name	Encrypted API credentials, site name	Low
Lovable	Site pulls from Rankoya (or Rankoya pings a webhook on the site)	Rankoya-issued API key	Hashed key, optional webhook URL and signing secret	Medium (one-time Content API)
Replit	Same as Lovable	Rankoya-issued API key	Same as Lovable	Low once Lovable ships
Make	Make scenario receives Rankoya events	Make custom webhook URL (phase 1); Rankoya API key in a Make custom app (phase 2)	Webhook URL + signing secret	Low, then Medium

Integration	Direction	Auth model	What a connection stores	Build effort
Zapier	Zap triggers on Rankoya events	Rankoya as an OAuth 2.0 provider + REST hooks	OAuth client, hashed access/refresh tokens, hook subscriptions	High
RSS feed	Any reader pulls a per-site feed	Unguessable token in the feed URL	Hashed feed token	Low

Where the code lives. Your frontend is Next 16 with a Laravel Sanctum API behind it (see the technical walkthrough). Every flow below runs its OAuth handshake, secret handling and provider calls in Next route handlers under `src/app/api/integrations/`, using server-only env vars. The finished connection is saved through the existing `POST /sites/{site}/publishing/connections`, so the Laravel side keeps owning storage, encryption and publishing jobs.

One mismatch to check first: the walkthrough (4.12) lists Wix under the existing connection form. If that form takes an API key today, treat section 3 as replacing it with OAuth.

2. Shared foundation (build once)

Build these six pieces before any single integration; every flow below assumes them.

Step 2.1: Route layout

```
src/app/api/integrations/
  wix/start/route.ts          GET  → redirect to Wix install
  wix/callback/route.ts       GET  ← Wix redirect, finish connection
  duda/connect/route.ts       POST credentials → verify → save
  keys/route.ts               POST issue key (Lovable, Replit, Make) · GET
list
  keys/[id]/route.ts          DELETE revoke
  webhooks/route.ts           POST register outbound webhook (Make, Lovable,
Replit)
  webhooks/[id]/test/route.ts POST send a signed test event
  rss/route.ts                POST create or rotate feed token
src/app/api/public/
  v1/articles/route.ts        GET  (API key) for Lovable, Replit, Make
  v1/hooks/route.ts           POST/DELETE (Zapier REST hooks)
  feeds/[token]/route.ts      GET  RSS XML
src/app/oauth/
  authorize/page.tsx          consent screen for Zapier
src/app/api/oauth/
  token/route.ts              POST code or refresh_token → tokens
src/lib/integrations/
  state.ts  backend.ts  crypto.ts  connectors/*.ts
```

Keep the split strict: `/api/integrations/*` is called by signed-in users from the Rankoya UI; `/api/public/*` and `/api/oauth/*` are called by outside tools and authenticate with keys or tokens, never the session cookie.

Step 2.2: Environment variables (server only)

None of these get a `NEXT_PUBLIC_` prefix. Add them to `.env.example` with empty values and to Vercel per environment.

```
APP_URL=https://app.rankoya.com
INTEGRATIONS_STATE_SECRET=        # 32+ random bytes, signs OAuth state
cookies
INTEGRATIONS_WEBHOOK_SECRET_KEY=  # wraps per-webhook signing secrets if Next
ever holds them
WIX_APP_ID=
WIX_APP_SECRET=
WIX_SHARE_URL_ID=                 # only while the Wix app is unlisted
ZAPIER_CLIENT_ID=
ZAPIER_CLIENT_SECRET=
BACKEND_INTERNAL_URL=             # Laravel URL reachable from Vercel; may
equal NEXT_PUBLIC_API_BASE_URL
BACKEND_SERVICE_TOKEN=            # Sanctum token of a service user, for
/api/public and /api/oauth
```

Step 2.3: Forward the user's session to Laravel

Route handlers read the `authToken` cookie and call Laravel with it, exactly as axios does in the browser. The backend still resolves the active workspace from the token, so a user can never connect a site outside their workspace.

```
// src/lib/integrations/backend.ts
import 'server-only';
import { cookies } from 'next/headers';

export async function backend<T>(path: string, init: RequestInit = {}):
Promise<T> {
  const token = (await cookies()).get('authToken')?.value;
  if (!token) throw new IntegrationError('unauthenticated', 401);
  const res = await fetch(`${process.env.BACKEND_INTERNAL_URL}${path}`, {
    ...init,
    headers: {
      Accept: 'application/json',
      'Content-Type': 'application/json',
      Authorization: `Bearer ${token}`,
      ...init.headers,
    },
    cache: 'no-store',
```

```
  });
  if (!res.ok) throw new IntegrationError(await res.text(), res.status);
  return res.json() as Promise<T>;
}
```

`/api/public/*` routes use the same helper with `BACKEND_SERVICE_TOKEN` instead of the cookie, plus the workspace and site that the key or token resolves to.

Step 2.4: Signed, short-lived OAuth state

You currently keep `rankoya:gsc-pending` and `rankoya:wpcom-pending` in sessionStorage. For flows owned by Next, keep that context in an httpOnly cookie instead, signed so it can't be edited, and checked against the `state` the provider echoes back.

```
// src/lib/integrations/state.ts
import 'server-only';
import { createHmac, randomBytes, timingSafeEqual } from 'node:crypto';

export type OAuthState = { provider: string; siteId: string; returnTo:
string; nonce: string; exp: number };

const sign = (body: string) =>
  createHmac('sha256',
process.env.INTEGRATIONS_STATE_SECRET!).update(body).digest('base64url');

export function createState(p: Omit<OAuthState, 'nonce' | 'exp'>) {
  const state: OAuthState = { ...p, nonce:
randomBytes(16).toString('base64url'), exp: Date.now() + 10 * 60_000 };
  const body = Buffer.from(JSON.stringify(state)).toString('base64url');
  return { state, cookie: `${body}.${sign(body)}` };
}

export function readState(cookie: string | undefined, returnedNonce: string |
null): OAuthState {
  const [body, sig] = (cookie ?? '').split('.');
  if (!body || !sig) throw new IntegrationError('missing_state', 400);
  const a = Buffer.from(sig), b = Buffer.from(sign(body));
  const ok = a.length === b.length && timingSafeEqual(a, b);
  const state = JSON.parse(Buffer.from(body, 'base64url').toString()) as
OAuthState;
```

```
  if (!ok || state.exp < Date.now() || state.nonce !== returnedNonce) throw
new IntegrationError('bad_state', 400);
  return state;
}
```

Set the cookie as `rk_oauth_<provider>` with `httpOnly`, `secure`, `sameSite: 'lax'` (the provider redirect is a top-level GET), `path: '/api/integrations'` and `maxAge: 600`. Delete it in the callback whatever the outcome. Only accept a `returnTo` that starts with a single `/`, so the callback can't become an open redirect.

Step 2.5: One connector interface

```
// src/lib/integrations/connectors/types.ts
export type Platform = 'wix' | 'duda' | 'lovable' | 'replit' | 'make' |
'zapier' | 'rss';

export interface Connector {
  platform: Platform;
  kind: 'push' | 'pull' | 'event';
  verify(credentials: Record<string, string>): Promise<{ ok: true; label:
string } | { ok: false; reason: string }>;
  publish?(article: ArticlePayload, credentials: Record<string, string>):
Promise<{ remoteId: string; url?: string }>;
}
```

Push connectors (Wix, Duda) implement `verify` and `publish`. If publishing jobs run in Laravel queues, port `publish` to PHP and keep only `verify` here; decide this once and write it at the top of the file.

Step 2.6: Backend contract

Ask the backend for these before starting; most flows block on them.

Endpoint	Needed by	Notes
`POST /sites/{site}/publishing/connections` accepting `platform` = wix, duda, lovable, replit, make, zapier, rss	All	`credentials` stored with Laravel's `encrypted` cast; never returned in `GET` responses

Endpoint	Needed by	Notes
`DELETE /sites/{site}/publishing/connections/{id}`	All	Already on the walkthrough's missing list; revoking keys and tokens depends on it
`POST /sites/{site}/api-keys`, `GET`, `DELETE /{id}`	Lovable, Replit, Make, RSS	Store a SHA-256 hash and a short display prefix; return the plain key once
`POST /sites/{site}/webhooks`, `DELETE /{id}`, `POST /{id}/test`	Make, Lovable, Replit, Zapier	Laravel signs and sends events from its queue on `article.published` and `article.updated`
`POST /internal/api-keys/resolve`	`/api/public/*`	Takes a key hash, returns workspace, site and scopes; service token only
OAuth server tables (Laravel Passport fits)	Zapier	Clients, auth codes, access and refresh tokens

3. Wix: app install + client-credentials tokens

Wix no longer hands you a refresh token to store. You register Rankoya as a self-managed Wix app, the user installs it on their site, and Rankoya keeps only the `instanceId`; every publish mints a fresh 4-hour access token from app ID + app secret + instance ID (Wix: Authenticate using OAuth).

Step 3.1: Create the Wix app (one-time, in the Wix dev center)

1. Create a self-managed app named Rankoya and copy its **App ID** and **App secret** into `WIX_APP_ID` and `WIX_APP_SECRET`.

2. Under Permissions, add **Manage Blog** (`SCOPE.DC-BLOG.MANAGE-BLOG`), which draft-post create and publish require (Wix: Publish Draft Post). Add read access to site members too, for the author lookup in step 3.5.
3. While the app is unlisted, create a share install link and keep its `shareUrlId` (the GUID the short link resolves to) in `WIX_SHARE_URL_ID` (Wix: Set Up the External Install Flow). Drop it once the app is listed in the App Market.

Step 3.2: Start route

The user clicks **Connect Wix** on Integrations (or in onboarding step 6). The button is a plain link to `/api/integrations/wix/start?site=<siteId>&returnTo=/sites/<siteId>/integrations` .

```
// src/app/api/integrations/wix/start/route.ts
import { NextResponse, type NextRequest } from 'next/server';
import { createState } from '@/lib/integrations/state';
import { safeReturnTo } from '@/lib/integrations/urls';

export async function GET(req: NextRequest) {
  const siteId = req.nextUrl.searchParams.get('site');
  if (!siteId || !req.cookies.get('authToken')) return
NextResponse.redirect(new URL('/sign-in', req.url));

  const { state, cookie } = createState({
    provider: 'wix', siteId,
    returnTo: safeReturnTo(req.nextUrl.searchParams.get('returnTo')),
  });

  const callback = new URL('/api/integrations/wix/callback',
process.env.APP_URL);
  callback.searchParams.set('state', state.nonce); // Wix preserves your own
query params

  const install = new URL('https://www.wix.com/app-installer');
  install.searchParams.set('appId', process.env.WIX_APP_ID!);
  if (process.env.WIX_SHARE_URL_ID) install.searchParams.set('shareUrlId',
process.env.WIX_SHARE_URL_ID);
  install.searchParams.set('postInstallationUrl', callback.toString());

  const res = NextResponse.redirect(install);
  res.cookies.set('rk_oauth_wix', cookie, {
    httpOnly: true, secure: true, sameSite: 'lax', path: '/api/integrations',
```

```
maxAge: 600,
  });
  return res;
}
```

Only the random nonce travels through Wix; the site ID and return path stay in the signed cookie. `URLSearchParams.set` encodes the callback once, which is what Wix expects.

Step 3.3: Verify `signedInstance`

After install, Wix redirects to the callback with `appId`, `tenantId`, `instanceId` and, on success, `signedInstance`. Wix says to verify `signedInstance` before trusting `instanceId`, and to treat a missing value as a failed or cancelled install. The parameter is `signature.data`, where the signature is an HMAC-SHA256 of the data part keyed with the app secret, base64url-encoded (Wix: About App Instances).

```
// src/lib/integrations/connectors/wix.ts
import 'server-only';
import { createHmac, timingSafeEqual } from 'node:crypto';

export function verifySignedInstance(signed: string): { instanceId: string }
{
  const [sig, data] = signed.split('.');
  const expected = createHmac('sha256',
process.env.WIX_APP_SECRET!).update(data).digest('base64url');
  const a = Buffer.from(sig), b = Buffer.from(expected);
  if (a.length !== b.length || !timingSafeEqual(a, b)) throw new
Error('wix_bad_signature');
  return JSON.parse(Buffer.from(data, 'base64url').toString());
}
```

Step 3.4: Callback route

```
// src/app/api/integrations/wix/callback/route.ts
export async function GET(req: NextRequest) {
  const q = req.nextUrl.searchParams;
  const fail = (reason: string, to = '/integrations') =>
    clear(NextResponse.redirect(new URL(`${to}?error=wix_${reason}`,
req.url)));

  let state;
```

```
  try { state = readState(req.cookies.get('rk_oauth_wix')?.value,
q.get('state')); }
  catch { return fail('state'); }

  const instanceId = q.get('instanceId'), signed = q.get('signedInstance');
  if (!instanceId || !signed) return fail('cancelled', state.returnTo);

  const verified = verifySignedInstance(signed);
  if (verified.instanceId !== instanceId) return fail('signature',
state.returnTo);

  const token = await wixToken(instanceId);                 // step 3.5
  const authorMemberId = await findOwnerMemberId(token);    // step 3.5
  await assertBlogInstalled(token);                         // step 3.5

  const conn = await backend<{ data: { id: string } }>(
    `/sites/${state.siteId}/publishing/connections`,
    { method: 'POST', body: JSON.stringify({
        platform: 'wix',
        credentials: { instance_id: instanceId, wix_site_id:
q.get('tenantId'), author_member_id: authorMemberId },
    }) },
  );
  await
backend(`/sites/${state.siteId}/publishing/connections/${conn.data.id}/test`,
{ method: 'POST' });
  return clear(NextResponse.redirect(new URL(`${state.returnTo}?
connected=wix`, req.url)));
}

const clear = (res: NextResponse) => { res.cookies.delete({ name:
'rk_oauth_wix', path: '/api/integrations' }); return res; };
```

Wrap the three Wix calls in `try/catch` and map each failure to its own error code (`wix_no_blog`, `wix_token`), so the Integrations screen can show a specific message instead of a generic one.

Step 3.5: Token, author and blog check

```
export async function wixToken(instanceId: string): Promise<string> {
  const res = await fetch('https://www.wixapis.com/oauth2/token', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      grant_type: 'client_credentials',
      client_id: process.env.WIX_APP_ID,
      client_secret: process.env.WIX_APP_SECRET,
      instance_id: instanceId,
    }),
  });
  if (!res.ok) throw new Error('wix_token');
  return (await res.json()).access_token as string; // valid 4 h; cache per
instance for ~3.5 h
}
```

- Send the token as `Authorization: <token>` with no `Bearer` prefix, as Wix's own REST examples do.
- For third-party apps, Create Draft Post requires a `memberId` (Wix: Create Draft Post). Look up the site owner's member record once at connect time, store it as `author_member_id`, and let the user change the author later in connection settings.
- `assertBlogInstalled` lists blog categories; a failure there means Wix Blog isn't on the site. Show "Add Wix Blog to your site, then connect again" rather than saving a broken connection.

Step 3.6: Publish

1. `POST https://www.wixapis.com/blog/v3/draft-posts` with `title`, `memberId`, `richContent`, `excerpt`, `heroImage` and SEO data. Body content must be Ricos JSON, not HTML, so convert Rankoya's article HTML first (Wix offers a Ricos conversion API; confirm the current endpoint when you build this).
2. If the connection is set to auto-publish, `POST /blog/v3/draft-posts/{draftPostId}/publish`; it returns the live `postId`. Republishing an edited article uses `PATCH /blog/v3/draft-posts/{id}` with `action: UPDATE_PUBLISH`.
3. Store `draftPostId` and `postId` on the Rankoya article so later edits update the same post instead of creating a duplicate. A single draft post is capped at 400 KB.

Step 3.7: Uninstall

Subscribe the Wix app to the **App Instance Removed** webhook and point it at a public route (`/api/public/wix/events`). Verify the webhook's JWT with the app's public key, then mark the matching connection as disconnected so publishing stops with a clear "Reconnect Wix" state.

4. Duda: API username, password and site name

Duda has no user-facing OAuth for this. Its Partner API uses HTTP Basic auth with an API username and password tied to the Duda account, sent to `https://api.duda.co/api`, and every request must carry a `User-Agent` header (Duda: API reference). Only Duda customers on custom or partner plans get these credentials (RightBlogger: Duda integration), so the connect form must say so up front.

Step 4.1: Connect form (UI)

Three fields plus one toggle, in the existing integration sheet:

Field	Notes
API username	From the Duda partner dashboard
API password	Masked input; never echoed back after save
Site name	Duda's site ID (also shown as the site alias in site details)
Publish immediately	Off by default: posts land as drafts in Duda's blog editor
Region	`api.duda.co` by default; `api.eu.duda.co` for EU accounts (@dudadev/api)

Helper line under the form: "Duda API access is available on Duda custom and partner plans. Not sure? Ask your Duda account manager."

Step 4.2: Connect route (verify, then save)

Credentials go from the browser to your own route handler once, over HTTPS, and are never stored in the browser.

```
// src/app/api/integrations/duda/connect/route.ts
import { z } from 'zod';

const Body = z.object({
  siteId: z.string().min(1),
  username: z.string().min(1),
  password: z.string().min(1),
  siteName: z.string().regex(/^[a-z0-9]+$/i),
  host: z.enum(['api.duda.co', 'api.eu.duda.co']).default('api.duda.co'),
  autoPublish: z.boolean().default(false),
});

export async function POST(req: NextRequest) {
  const body = Body.parse(await req.json());
  const auth = 'Basic ' +
Buffer.from(`${body.username}:${body.password}`).toString('base64');

  const res = await
fetch(`https://${body.host}/api/sites/multiscreen/${body.siteName}`, {
    headers: { Authorization: auth, Accept: 'application/json', 'User-Agent':
'Rankoya/1.0' },
  });
  if (res.status === 401 || res.status === 403) return Response.json({ error:
'duda_auth' }, { status: 422 });
  if (res.status === 404) return Response.json({ error: 'duda_site_not_found'
}, { status: 422 });
  if (!res.ok) return Response.json({ error: 'duda_unavailable' }, { status:
502 });

  const conn = await backend<{ data: { id: string } }>
(`/sites/${body.siteId}/publishing/connections`, {
    method: 'POST',
    body: JSON.stringify({
      platform: 'duda',
      credentials: { username: body.username, password: body.password,
site_name: body.siteName, host: body.host },
      settings: { auto_publish: body.autoPublish },
    }),
  });
  return Response.json({ id: conn.data.id });
}
```

- The verify call is a Get Site on the given site name: it proves the credentials work and that this account can reach that site.
- Never log the request body or the `Authorization` header. Add a redaction rule for `password` and `Basic ` to your logger before this ships.
- The UI shows the three error codes as field-level messages: wrong credentials, site not found, Duda unavailable.

Step 4.3: Publish

Use Duda's **Import Blog Post** API, which exists for exactly this case: pushing posts written outside Duda, including AI-generated ones, into a site's blog (Duda: Blog APIs announcement). Imported posts arrive as drafts; when `auto_publish` is on, call Duda's publish-post endpoint right after. Store the returned post ID on the Rankoya article so edits update the same post. Confirm the exact paths and body fields in Duda's API reference when you write the connector, since the blog endpoints aren't in this guide's sources.

Step 4.4: Later option: Duda App Store app

If many target users are on standard Duda plans, list Rankoya as a Duda App Store app. App installs receive their own authorization token and a per-site `api_endpoint`, which the SDK passes on each call (@dudadev/api). This is a second, OAuth-like path you can add later behind the same `platform: 'duda'` connector; the credentials form stays for agencies.

5. Lovable and Replit: Rankoya Content API + site key

Neither platform has a CMS you can push posts into: a Lovable or Replit site is code the user owns. So Rankoya issues a site-scoped API key, exposes a small read API, and hands the user a ready-made prompt that makes their AI builder add a blog backed by that API. RankYak takes the same route for custom sites: an API to fetch articles and to report each article's live URL back (RankYak: API integration).

Step 5.1: Issue a site key

```
// src/app/api/integrations/keys/route.ts
const Body = z.object({
  siteId: z.string(),
  platform: z.enum(['lovable', 'replit', 'make', 'custom']),
```

```
  name: z.string().max(60),
});

export async function POST(req: NextRequest) {
  const { siteId, platform, name } = Body.parse(await req.json());
  // Laravel generates the key, stores sha256(key) + the first 10 chars,
returns the plain key once.
  const key = await backend<{ data: { id: string; key: string; prefix: string
} }>(
    `/sites/${siteId}/api-keys`,
    { method: 'POST', body: JSON.stringify({ name, scopes: ['articles:read',
'articles:report_url'], platform }) },
  );
  await backend(`/sites/${siteId}/publishing/connections`, {
    method: 'POST',
    body: JSON.stringify({ platform, credentials: { api_key_id: key.data.id }
}),
  });
  return Response.json(key.data, { headers: { 'Cache-Control': 'no-store' }
});
}
```

- Key format: `rk_live_` + 32 random bytes in base64url. The prefix lets you and GitHub-style secret scanners recognise a leaked key.
- Show the key once in a copy field with "You won't see this key again". Afterwards the UI lists keys by name, prefix, created date and last-used date, with **Revoke**.
- One key per integration, so revoking the Lovable key never breaks Make.

Step 5.2: Public Content API

Two endpoints are enough for a blog, plus one for reporting live URLs:

Method and path	Returns or does
`GET /api/public/v1/articles?status=published&page=1&per_page=20`	List: id, slug, title, excerpt, hero image, published_at, updated_at
`GET /api/public/v1/articles/{slug}`	One article: HTML body, meta title, meta description, canonical, schema JSON-LD, FAQ

Method and path	Returns or does
`POST /api/public/v1/articles/{id}/live-url` `{ url }`	Records where the article went live; needed for rank tracking and internal links

```
// src/lib/integrations/api-key-auth.ts
export async function requireApiKey(req: NextRequest, scope: string) {
  const raw = req.headers.get('x-rankoya-key') ??
req.headers.get('authorization')?.replace(/^Bearer /, '');
  if (!raw?.startsWith('rk_live_')) throw new IntegrationError('missing_key',
401);
  const hash = createHash('sha256').update(raw).digest('hex');
  const ctx = await serviceBackend<{ siteId: string; scopes: string[] }>
('/internal/api-keys/resolve', {
    method: 'POST', body: JSON.stringify({ hash }),
  });
  if (!ctx.scopes.includes(scope)) throw new IntegrationError('forbidden',
403);
  return ctx;
}
```

Responses carry `Cache-Control: public, s-maxage=300, stale-while-revalidate=3600` on reads, and a per-key rate limit (start at 60 requests a minute) to protect the backend. Add CORS only if you choose to allow browser calls; the prompt below keeps the key server-side, so you shouldn't need it.

Step 5.3: Signed webhook for instant updates (optional)

If the user's site has a server function, they can register its URL so Rankoya notifies it on `article.published` and `article.updated`, and the site can rebuild or refresh its cache. Laravel sends these from its queue; every request carries:

```
X-Rankoya-Event: article.published
X-Rankoya-Timestamp: 1730880000
X-Rankoya-Signature: v1=<hex hmac_sha256(secret, timestamp + "." + raw_body)>
```

Receivers reject anything older than 5 minutes or with a bad signature. Retry with backoff on non-2xx (1 min, 5 min, 30 min, 2 h, 6 h), then mark the webhook failing and notify the user.

Step 5.4: The setup prompt (Lovable)

After issuing the key, the Lovable card shows three numbered steps and a **Copy prompt** button:

1. In Lovable, open your project's secrets and add `RANKOYA_API_KEY` with the key. Lovable's docs ask you to enter keys through the Add secret form, not the chat, and to avoid secret names starting with `LOVABLE_`, which are reserved (Lovable: API keys, Lovable: any API).
2. Paste the prompt into the project chat.
3. Publish, then paste your live blog URL back into Rankoya to finish.

The prompt text, generated with the user's API base URL filled in:

```
Add a blog to this site backed by the Rankoya Content API.

- Read the API key from the secret RANKOYA_API_KEY. Call the API only from
server-side code; never expose the key to the browser.
- List endpoint: GET https://app.rankoya.com/api/public/v1/articles?
status=published&page={n} with header x-rankoya-key.
- Article endpoint: GET
https://app.rankoya.com/api/public/v1/articles/{slug}.
- Pages: /blog (paginated list with title, excerpt, image, date) and
/blog/{slug} (render the HTML body; set <title>, meta description, canonical
and the JSON-LD from the response).
- Generate /sitemap.xml entries for every article.
- After an article page is live, POST its URL to
/api/public/v1/articles/{id}/live-url once.
```

SEO warning to show on the card: older Lovable apps are React + Vite single-page apps (Lovable: any API). If the blog renders only in the browser, crawlers and AI engines may see an empty page, which defeats the point of Rankoya. Recommend prerendering or server rendering for `/blog/*`, and test with "View source" that article text is in the HTML.

Step 5.5: Replit

Same key, same API, same webhook; only the setup copy changes. The user stores `RANKOYA_API_KEY` in Replit's Secrets and pastes a prompt for Replit Agent. Replit apps usually have a real server (Express, Next.js, Flask), so the prompt can ask for server-rendered `/blog` routes and a webhook receiver that clears the cache on `article.updated`.

Step 5.6: When is it "connected"?

Mark the connection **Waiting for first request** after the key is issued, **Connected** when the key is first used, and **Verified** once a live URL is reported and Rankoya's crawler finds the article text in that page's HTML. That last check is what proves the SEO warning above was handled.

6. Make and Zapier: Rankoya as the provider

For Make and Zapier, Rankoya is the service being connected to, so you build the auth server side, not a client. Both need the same core: a way to authenticate (API key or OAuth), an endpoint to subscribe and unsubscribe a webhook URL, and events fired on `article.published`. RankYak's Make app is exactly this shape: one instant trigger, "Watch published articles" (Make: RankYak integration).

Step 6.1: Shared hook subscription API

Build this once; Make, Zapier and any custom tool use it.

Method and path	Auth	Body or result
`POST /api/public/v1/hooks`	API key or OAuth token	`{ target_url, event }` → `{ id }`
`DELETE /api/public/v1/hooks/{id}`	Same	204
`GET /api/public/v1/me`	Same	`{ workspace, site, email }`; used as the connection test
`GET /api/public/v1/articles?per_page=3`	Same	Sample items so the builder can show real fields

Rules for `target_url`: HTTPS only, and the host must belong to the platform (`hook.*.make.com`, `hooks.zapier.com`) when the subscription comes from that platform's app. When a delivery gets **410 Gone**, delete the subscription; any other failure retries with the schedule from step 5.3.

Event payload, same for every consumer:

```
{
  "event": "article.published",
  "id": "evt_01J...",
  "created_at": "2026-11-06T09:00:00Z",
  "site": { "id": "...", "domain": "example.com" },
  "article": { "id": "...", "slug": "...", "title": "...", "html": "...",
"meta_title": "...", "meta_description": "...", "hero_image_url": "...",
"live_url": null }
}
```

Step 6.2: Make, phase 1 (ships in a day)

1. The user creates a scenario starting with Make's **Custom webhook** module and copies its URL.
2. In Rankoya's Make card they paste the URL and click **Send test event**.
3. Your route `POST /api/integrations/webhooks` validates the host, asks Laravel to create the subscription with a new signing secret, then calls `/webhooks/{id}/test`. Make captures the payload structure from that test.
4. Show the signing secret once with a note: "Optional: verify `X-Rankoya-Signature` with a filter in your scenario."

No OAuth and no Make app review needed, so this unblocks users now.

Step 6.3: Make, phase 2 (custom app)

Build a Rankoya app in Make's custom apps editor:

1. **Connection:** one field, API key, issued from Rankoya's Make card (step 5.1 with `platform: 'make'`). Its validation call is `GET /api/public/v1/me` with header `x-rankoya-key`.
2. **Webhook:** dedicated, attached. Make's attach procedure posts `{{webhook.url}}` to your subscribe endpoint and must save your returned id into `response.data`, because the detach procedure reads it from there (Make: attached webhooks).

```
{
  "attach": {
    "url": "https://app.rankoya.com/api/public/v1/hooks",
    "method": "POST",
    "headers": { "x-rankoya-key": "{{connection.apiKey}}" },
    "body": { "target_url": "{{webhook.url}}", "event": "article.published"
},
    "response": { "data": { "externalHookId": "{{body.id}}" } }
  },
  "detach": {
    "url":
"https://app.rankoya.com/api/public/v1/hooks/{{webhook.externalHookId}}",
    "method": "DELETE",
    "headers": { "x-rankoya-key": "{{connection.apiKey}}" }
  }
}
```

3. **Modules:** instant trigger "Watch published articles", action "Report live URL", and the universal "Make an API call" module.
4. Two behaviours to plan for, reported by a team that shipped a Make app: attach runs when the scenario first runs, not when the webhook is created; and a webhook outlives its scenario until the user deletes it under Webhooks, which is the only thing that calls detach (WaAPI Make app notes). So subscriptions must be idempotent, and stale ones must be cleaned up by the 410 rule above.

Step 6.4: Zapier: Rankoya as an OAuth 2.0 provider

Zapier apps that act for many users should use OAuth v2. You need three pieces; if you run them in Laravel (Passport fits), the Next routes below just proxy.

a. Register the client. In the Zapier developer platform, set authorization URL `https://app.rankoya.com/oauth/authorize`, token URL `https://app.rankoya.com/api/oauth/token`, and refresh enabled. Copy Zapier's redirect URI, which has the form `https://zapier.com/dashboard/auth/oauth/return/App<id>CLIAPI/` (Zapier community thread), into the client record. Put the client ID and secret in `ZAPIER_CLIENT_ID` and `ZAPIER_CLIENT_SECRET`.

b. Authorize page (`src/app/oauth/authorize/page.tsx`). `src/proxy.ts` already sends signed-out visitors to sign-in with `?next=`, so this page always has a session.

1. Validate `client_id`, `response_type=code`, and that `redirect_uri` equals the registered URI exactly; on any mismatch show an error and never redirect.
2. Show a consent card: "Zapier wants to read your articles and receive publish events for" + a site picker limited to the active workspace + **Allow** / **Cancel**.
3. On **Allow**, a server action asks the backend to create a single-use code (60 s lifetime, bound to user, site, client and redirect URI) and redirects to `redirect_uri?code=…&state=<state as received>`. On **Cancel**, redirect with `error=access_denied&state=…`.

c. Token route.

```
// src/app/api/oauth/token/route.ts
export async function POST(req: NextRequest) {
  const form = await req.formData(); // Zapier sends form-encoded or JSON;
accept both in production
  const grant = form.get('grant_type');
  if (form.get('client_id') !== process.env.ZAPIER_CLIENT_ID ||
      form.get('client_secret') !== process.env.ZAPIER_CLIENT_SECRET) {
    return Response.json({ error: 'invalid_client' }, { status: 401 });
  }
  const body = grant === 'authorization_code'
    ? { grant, code: form.get('code'), redirect_uri: form.get('redirect_uri')
}
    : grant === 'refresh_token'
      ? { grant, refresh_token: form.get('refresh_token') }
      : null;
  if (!body) return Response.json({ error: 'unsupported_grant_type' }, {
status: 400 });

  const tokens = await serviceBackend('/internal/oauth/token', { method:
'POST', body: JSON.stringify(body) });
  return Response.json(tokens, { headers: { 'Cache-Control': 'no-store' } });
  // → { access_token, refresh_token, token_type: 'Bearer', expires_in: 3600
}
}
```

Compare secrets with `timingSafeEqual` in the real code. Store only hashes of access and refresh tokens; rotate the refresh token on each use, and revoke the whole family if an old one is replayed.

d. Zapier app definition. Connection test: `GET /api/public/v1/me` with `Authorization: Bearer {{bundle.authData.access_token}}`. Trigger "New published article" as a REST hook: `performSubscribe` posts `bundle.targetUrl` to `/hooks`, and `performUnsubscribe` deletes using the id saved in `bundle.subscribeData` (Zapier: bundle reference). `performList` returns the sample articles so users can map fields.

e. Show it in Rankoya. The Zapier card lists active connections (from the OAuth grants), each with site, created date and **Revoke**, which deletes the grant and its hook subscriptions.

7. RSS feed: one secret URL per site

RSS has no auth handshake: the credential is an unguessable token in the feed URL, which the user can rotate. It's the cheapest integration on the list and doubles as a fallback for any tool with an RSS reader, including Make's and Zapier's built-in RSS modules and Duda's own RSS blog import (Duda: Blog APIs).

Step 7.1: Create or rotate the feed URL

```
// src/app/api/integrations/rss/route.ts
export async function POST(req: NextRequest) {
  const { siteId, rotate } = z.object({ siteId: z.string(), rotate:
z.boolean().default(false) }).parse(await req.json());
  // Laravel stores sha256(token) on the site's rss connection; rotating
replaces it, killing the old URL.
  const { data } = await backend<{ data: { token: string } }>
(`/sites/${siteId}/rss-token`, {
    method: 'POST', body: JSON.stringify({ rotate }),
  });
  return Response.json({ url:
`${process.env.APP_URL}/api/public/feeds/${data.token}.xml` }, { headers: {
'Cache-Control': 'no-store' } });
}
```

Use 32 random bytes in base64url for the token. Because a URL leaks more easily than a header (logs, browser history, shared screenshots), the card shows the URL masked with **Copy** and **Rotate**, and Rotate asks for confirmation: "Any tool using the old URL will stop receiving articles."

Step 7.2: Serve the feed

```
// src/app/api/public/feeds/[token]/route.ts
export async function GET(req: NextRequest, { params }: { params: Promise<{
token: string }> }) {
  const token = (await params).token.replace(/\.xml$/, '');
  const hash = createHash('sha256').update(token).digest('hex');
  const feed = await serviceBackend<FeedData | null>
(`/internal/feeds/${hash}`).catch(() => null);
  if (!feed) return new Response('Not found', { status: 404 });

  const etag = `"${feed.lastModified}"`;
  if (req.headers.get('if-none-match') === etag) return new Response(null, {
status: 304 });

  return new Response(renderRss(feed), {
    headers: {
      'Content-Type': 'application/rss+xml; charset=utf-8',
      ETag: etag,
      'Last-Modified': new Date(feed.lastModified).toUTCString(),
      'Cache-Control': 'public, s-maxage=600, stale-while-revalidate=3600',
      'X-Robots-Tag': 'noindex',
    },
  });
}
```

This route needs no session, so add `/api/public/` to the paths `src/proxy.ts` skips; otherwise readers get redirected to sign-in.

Step 7.3: Feed contents

- RSS 2.0 with the `content` namespace: `<content:encoded>` carries the full article HTML in CDATA, and `<description>` carries the excerpt.
- Per item: `title`, `link` (live URL if reported, else Rankoya's preview URL), `guid isPermaLink="false"` set to the Rankoya article ID so readers never import a post twice, `pubDate` in RFC 822, `category` per topic cluster, and an `enclosure` for the hero image.
- Channel: site name, site URL, language, `atom:link rel="self"`, and `ttl` of 60.
- Latest 50 published articles, newest first. Escape everything outside CDATA, and split any `]]>` inside article HTML.

- `X-Robots-Tag: noindex` keeps search engines from indexing the feed as duplicate content of the user's own site.

Step 7.4: Connected state

Show **Active** with "Last read 3 hours ago" from the newest feed request Laravel logged, or **Not read yet**. A feed nobody has read in 30 days gets a gentle prompt to rotate or remove it.

8. UI wiring, testing, security and rollout

Ship in the order below: each step reuses what the one before it built, and the first two unblock users in under a week.

Step 8.1: Rollout order

Order	Integration	Depends on	Rough size
1	Shared foundation (section 2) + `DELETE` connection endpoint	Backend contract	2–3 days
2	RSS feed	Foundation	1 day
3	Make phase 1 (custom webhook URL)	Webhook delivery in Laravel	1–2 days
4	Duda	Foundation	2 days
5	Lovable, then Replit	Site keys + Content API	3–4 days, then 1 day
6	Wix	Wix app approved for Manage Blog	3–4 days
7	Make phase 2 (custom app)	Hook subscription API	2–3 days
8	Zapier	OAuth provider + hook API; Zapier review before public listing	5–8 days

Step 8.2: Integrations screen and onboarding

- One card per platform, using the existing integration sheet. The card's action depends on the auth model: **Connect** (redirect: Wix), a form (Duda), **Generate key** (Lovable, Replit, Make phase 2), **Add webhook URL** (Make phase 1), **Copy feed URL** (RSS), or **Open Zapier** (Zapier, which starts from Zapier's side).
- Card states, shared by all seven: Not connected · Waiting for first request · Connected · Needs attention (failing deliveries, revoked token, Wix app removed) · Disconnected.
- Redirect flows land back on the Integrations page with `?connected=<platform>` or `?error=<code>`. Read them once, show a toast, invalidate the React Query `publishing/connections` key, then strip the params with `router.replace`.
- Onboarding step 6 lists the new platforms too, filtered by the detected website platform from `PATCH sites/{site}/platform`, so a Wix site sees Wix first.
- Keep the v1/v2 rules: every label and error message goes through i18n with Arabic strings, layout uses logical properties for RTL, and the cards are checked at 375 and 1440 px.
- Add a **Disconnect** button to every card once the `DELETE` endpoint exists. For Lovable, Replit, Make and RSS it also revokes the key, webhook or feed token.

Step 8.3: `src/proxy.ts` changes

- Skip the session check for `/api/public/*`, `/api/oauth/*` and `/api/public/wix/events`; these authenticate with keys, tokens or signatures.
- Keep `/oauth/authorize` behind sign-in, so the existing `?next=` redirect brings the user back to the consent screen.
- Extend `requireApiKey` (step 5.2) to also accept OAuth access tokens (give them a distinct prefix such as `rk_oat_`), so the public API serves Make keys and Zapier tokens through one check.

Step 8.4: Testing

Integration	Test before release
Wix	Install on a test site with Wix Blog: draft created, publish works, edit updates the same post. Cancel mid-install → `wix_cancelled`. Tampered `signedInstance` → rejected. Uninstall → card shows Needs attention

Integration	Test before release
Duda	Wrong password → field error, nothing saved. Wrong site name → `duda_site_not_found`. EU host works. Credentials never appear in logs or `GET` responses
Lovable / Replit	Prompt builds a working `/blog` on a fresh project. Article text is present in "View source". Revoked key → 401 within a minute
Make	Test event arrives and maps. Phase 2: deleting the webhook in Make removes the subscription
Zapier	Full OAuth dance, refresh after expiry, replayed refresh token revokes the family, unsubscribe on Zap off, 410 cleanup
RSS	Validates in a feed validator. Rotate kills the old URL. `If-None-Match` returns 304

OAuth redirects need public HTTPS (Wix requires HTTPS for the callback), so test redirect flows on a Vercel preview deployment with its own Wix test app and Zapier dev client, never against production credentials.

Step 8.5: Security checklist

- [ ] All provider secrets are server-only env vars; a build check fails if any `WIX_`, `ZAPIER_` or `*_SECRET` name appears with `NEXT_PUBLIC_`
- [ ] OAuth state is signed, single-use, expires in 10 minutes, and `returnTo` accepts only same-site relative paths
- [ ] Wix `signedInstance` verified before trusting `instanceId`; Wix webhooks verified with the app's public key
- [ ] Zapier `redirect_uri` checked by exact match; auth codes single-use and 60-second
- [ ] Third-party credentials (Duda) stored with Laravel's `encrypted` cast; Rankoya-issued keys, tokens and feed tokens stored only as SHA-256 hashes
- [ ] Secrets shown once, with `Cache-Control: no-store` on every response that carries one
- [ ] Logger redacts `password`, `Authorization`, `x-rankoya-key`, `client_secret`, `code` and `refresh_token`

- [ ] Outbound webhooks HTTPS-only, signed with timestamp, and blocked from private IP ranges to prevent SSRF
- [ ] Per-key and per-IP rate limits on `/api/public/*` and `/api/oauth/token`
- [ ] Every connection change (create, rotate, revoke) written to an audit log with user and time

Step 8.6: Open questions for the team

- Do publishing jobs run in Laravel queues or in Next? Section 2.5 assumes Laravel; if Next, you also need a job runner on Vercel.
- Is Wix currently connected with an API key form (walkthrough 4.12)? If yes, plan a migration message for existing Wix users.
- Which plans include API keys, Make and Zapier? Gate the cards before launch, not after.

Sources

- Wix: Authenticate using OAuth
- Wix: Set Up the External Install Flow
- Wix: About App Instances
- Wix: Create Draft Post and Publish Draft Post
- Duda: API reference and Blog APIs announcement
- @dudadev/api package
- RightBlogger: Duda integration
- Lovable: API keys and Lovable: any API
- RankYak: API integration
- Make: RankYak integration
- Make: attached webhooks
- WaAPI Make app notes
- Zapier: bundle reference
- Zapier community: OAuth redirect URI